

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

# Load dataset
df=pd.read_csv("data/housing.csv")
df.dropna(inplace=True)

# Create feature
df["avg_rooms"]=df["total_rooms"]/df["households"]
X=df["avg_rooms"].values
y=df["median_house_value"].values

# Split dataset
X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=0.2,random_state=42)

# Feature scaling
mean=X_train.mean()
std=X_train.std()
X_train_norm=(X_train-mean)/std
X_test_norm=(X_test-mean)/std
print("Training Data Mean :",mean)
print("Training Data Std  :",std)

# Initialize parameters
b0=0
b1=0
learning_rate=0.01
iteration=1000
n=len(X_train_norm)
cost=[]

# Gradient Descent
for i in range(iteration):
    y_pred=b0+b1*X_train_norm
```

```

mse=np.mean((y_train-y_pred)**2)
cost.append(mse)

db0=(-2/n)*np.sum(y_train-y_pred)
db1=(-2/n)*np.sum(X_train_norm*(y_train-y_pred))
b0=b0-learning_rate*db0
b1=b1-learning_rate*db1

# Prediction using Gradient Descent
y_pred_gd=b0+b1*X_test_norm

# Gradient Descent graph
plt.scatter(X_test_norm,y_test,color='blue',label='Actual Data')
plt.plot(X_test_norm,y_pred_gd,color='red',label='Regression Line')
plt.xlabel("Average number of rooms")
plt.ylabel("Median house value")
plt.title("Gradient Descent Graph")
plt.legend()
plt.show()

# Cost convergence graph
plt.plot(range(iteration),cost,color='green')
plt.xlabel("Iteration")
plt.ylabel("MSE")
plt.title("Cost Convergence Graph")
plt.grid(True)
plt.show()

# Gradient Descent results
print("\n--- Gradient Descent Results ---")
print("MSE :",mean_squared_error(y_test,y_pred_gd))
print("R2 Score :",r2_score(y_test,y_pred_gd))

# Least Squares Method
model=LinearRegression()
model.fit(X_train.reshape(-1,1),y_train.reshape(-1,1))
y_predict=model.predict(X_test.reshape(-1,1))

MSE=mean_squared_error(y_test,y_predict)
R2=r2_score(y_test,y_predict)

```

```

# Least Squares results

print("\n--- Least Squares Results ---")

print("MSE :",MSE)

print("R2 Score :",R2)

print("Slope (m) :",model.coef_[0])

print("Intercept (c) :",model.intercept_)


# Regression graph

m=model.coef_[0]

c=model.intercept_

plt.scatter(X_test,y_test,color='blue',label='Actual Data')

x_line=np.sort(X_test)

y_line=m*x_line+c

plt.plot(x_line,y_line,color='red',label='Regression Line')

plt.title("Simple Linear Regression: Average Rooms vs Median House Value")

plt.xlabel("Average number of rooms")

plt.ylabel("Median house value")

plt.legend()

plt.show()

```

```

Training Data Mean : 5.424985927537479
Training Data Std  : 2.359803695972903

```

```

--- Gradient Descent Results ---
MSE : 13554754931.142567
R2 Score : 0.008806161819007507

```

```

--- Least Squares Results ---
MSE : 13554754931.240023
R2 Score : 0.008806161811881208
Slope (m) : [7878.56487622]
Intercept (c) : [163903.29651521]

```

